Comparar la implementación de un diccionario de datos físico en diferentes SGBD

Informe

Realizado por: María Sofia Pabón Pabón

Grupo: G101 Bases de Datos

Docente: Oralia Cortes Grajales

Institución Universitaria Pascual Bravo

25 de octubre de 2026

Introducción

La comunidad estudiantil de la institución universitaria Pascual Bravo requiere una plataforma digital que promueva e incentive la interacción entre estudiantes, que se facilite la creación de perfiles, conformación de grupos de estudio, publicación de contenidos de diferentes tipos, seguimiento de eventos dentro del campus y la posibilidad de mentorías

En las fases anteriores del proyecto de la red se diseñaron los modelos: conceptual, lógico y físico y en esta parte de la tarea numero 4, el objetivo como tal es implementar la estructura física de la base de datos en tres sistemas de gestión de base de datos, mencionados en la plataforma de la universidad (PostgreSQL 17, MySQL 9.0 y SQLite 3)

Además, se incluye el soporte para datos semiestructurados (NoSQL / JSON) con el propósito de capacitar al sistema para contextos reales de Big Data e internet de las cosas (IoT)

Justificación de los tres SGBD seleccionados

Para dar el soporte de la red social pascualina y realizar un análisis comparativo del rendimiento se seleccionaron los siguientes tres motores:

PostgreSQL 17 (SGBD Principal -Relacional): Se eligió porque este sería el motor de principal de la red, se destaca por ser muy seguro y por manejar muy bien las relaciones entre tablas y soporta JSONB un formato para guardar y buscar datos que se cambian seguido

MySQL 9.0 (Sistema Web): Elegido porque es el sistema de base de datos relacionales de código abierto más usado, soporta datos en formato JSON de forma nativa y la valida de manera automática, lo que lo hace muy eficiente para aplicaciones de web rápidas

SQLite 3 (SGBD- Portable/IoT): Es una base de datos muy liviana que no necesita un servidor y guarda todo en un solo archivo, entonces se eligió para probar cómo funciona la red en aplicaciones móviles o dispositivos mas pequeños, lo que permite también guardar datos JSON

Justificación de tipo de datos NoSQL (JSON) PARA BIG DATA E IoT

Un modelo relacional clásico dificulta el almacenamiento de datos altamente dinámicos debido a su rigidez por eso se inserto el tipo de dato semiestructurado JSON en dos tablas estratégicas

Un ejemplo del objeto JSON:


```
{
  "gps": {"latitud": 6.2442, "longitud": -75.5812},
  "sensor_aforo_id": "SEN-102",
  "aforo_maximo": 30,
  "temperatura_celsius": 22.5
}
```


Explicación: Lo que hace es que sirve para guardar en la base de datos información de un salón o evento como: la ubicación geográfica exacta (GPS), el codigo del sensor que mide la capacidad del salón (sensor_aforo_id": "SEN-102) el límite máximo de personas permitidas en determinado espacio (30 personas, aforo_maximo) la temperatura actual del salón (22.5°C, temperatura_celsius)

Ejemplo de Uso Big Data: Tabla publicación (Columna metadatos_bigdata)


```
{
  "dispositivo": "Android App v2.1",
  "ip_origen": "192.168.1.15",
  "tiempo_lectura_seg": 45,
  "etiquetas_dinamicas": ["programacion", "examen", "pascualino"]
}
```


Explicación: Este permite registrar la dirección IP, tipo de dispositivo móvil o navegador, tiempo de lectura, y los hashtags añadidos en el tiempo de ejecución, estos metadatos son muy importantes para alimentar los motores de recomendaciones

Diccionario de Datos Genérico (Estructura Lógica): Este diccionario describe la estructura general de la Red social pascualina derivada del diagrama Entidad-Relación y el proceso de normalización:

- estudiante: Guarda la información básica de los usuarios (su código único, nombres, correo institucional y contraseña)
- perfil: Contiene la información complementaria del estudiante (como lo es la biografía, foto, carrera)
- publicacion: Almacena los mensajes e interacciones de los usuarios, incluyendo metadatos dinámicos
- evento: Registra las actividades académicas y culturales, los datos de ubicación e IoT del campus.

Dificultades encontradas con sus Ventajas y Desventajas

SGBD	Ventajas	Desventajas o dificultades
PostgreSQL 17	Es muy rápido para buscar información dentro de los archivos JSON y mantiene la información muy bien organizada y segura	Es un poco más estricto a la hora de escribir el código y consume más memoria en el computador
MySQL 9.0	muy popular y fácil de conectar con páginas web. Además, revisa automáticamente que los datos JSON estén bien escritos.	La forma de escribir el código cambia en algunas partes respecto a los otros programas
SQLite 3	Es súper liviano y no necesita instalar un servidor pesado es Ideal para aplicaciones móviles en celulares	Guarda los datos JSON como texto simple, y tiene menos opciones avanzadas para controlar los tipos de datos

Las dificultades encontradas durante la realización del trabajo serian: que cada programa usa palabras distintas y hacer que los IDs aumenten solos, como ejemplo PostgreSQL usa SERIAL, MySQL usa AUTO_INCREMENT y SQLite usa AUTOUNCREMENT

Otra dificultad fue activar las relaciones en SQLite ya que vienen por defecto las conexiones entre las tablas, apagadas así que se tuvo que activar manualmente la

opción para que al borrar algún dato no queden registros por ahí sueltos asimismo PRAGMA(FK) = ON para asegurar la ejecución de las reglas ON DELETE CASCADE.

Conclusiones

La implementación de la tarea numero 4 me permitió comprender que los sistemas de bases de datos no son solamente ejecutados en alguna SGBD en específico si no que depende de los requerimientos, en este proyecto la integración de columnas JSON dentro de la estructura relacional creo que representa una solución estable para aplicaciones de la vida real que deben gestionar volúmenes de datos semiestructurados como lo son el Big Data e IoT.

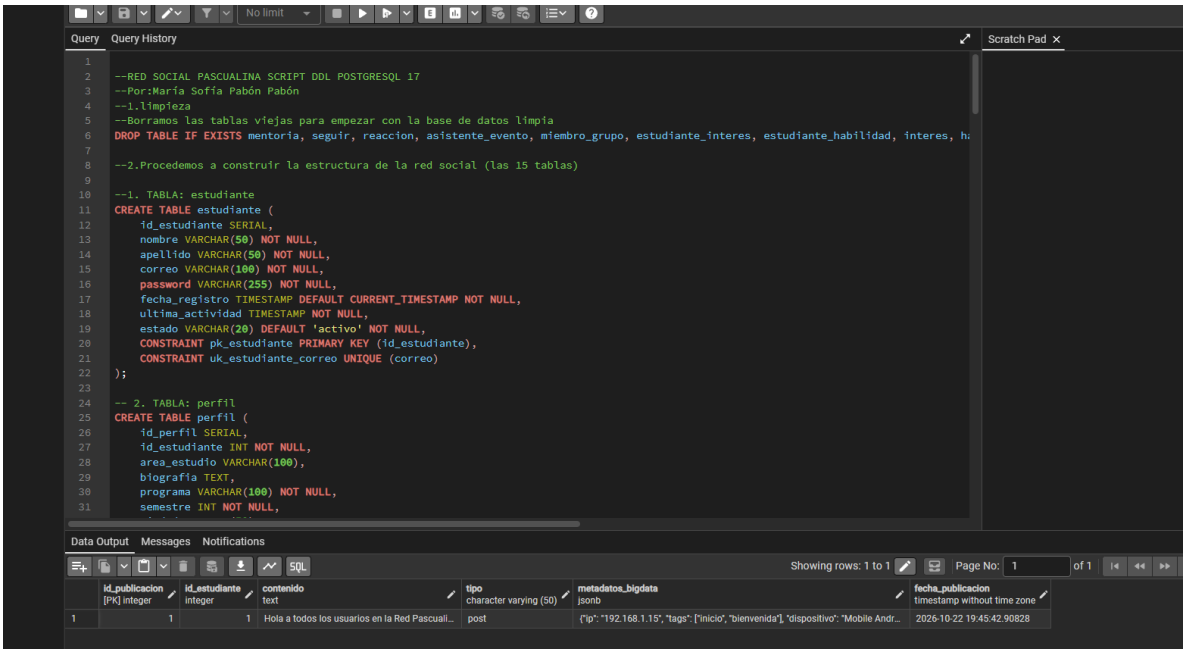
En lo personal nunca había hecho este tipo de cosas así que es un gran aprendizaje y reto ir avanzando por pasos para llegar a una solución bien completada.

Aprendí a identificar las particularidades de sintaxis de cada programa y sus tipos de datos equivalentes, por último, creo que me dio una visión critica para elegir el SGBD correcto, según las exigencias del proyecto.

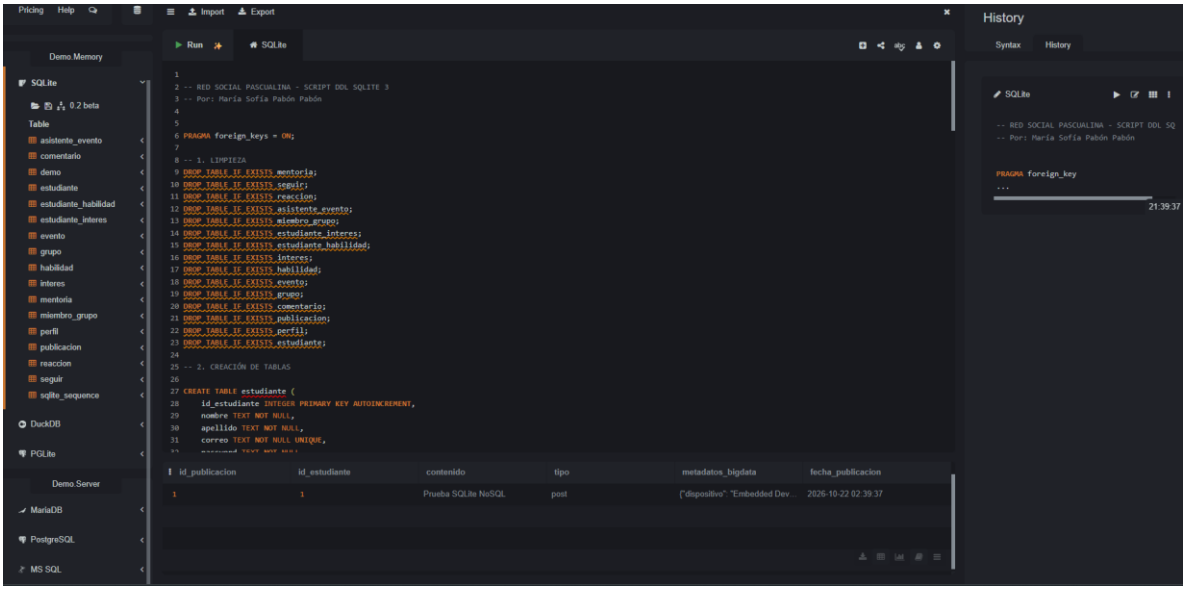
para recomendaciones de trabajos futuros seria aprovechar la base de datos en SQLite para desarrollar una aplicación en celulares que funcione en todo momento ya que creo que de esa forma seria más usada y más común entre los estudiantes del pascual bravo.

Vista de los SGBD implementados:

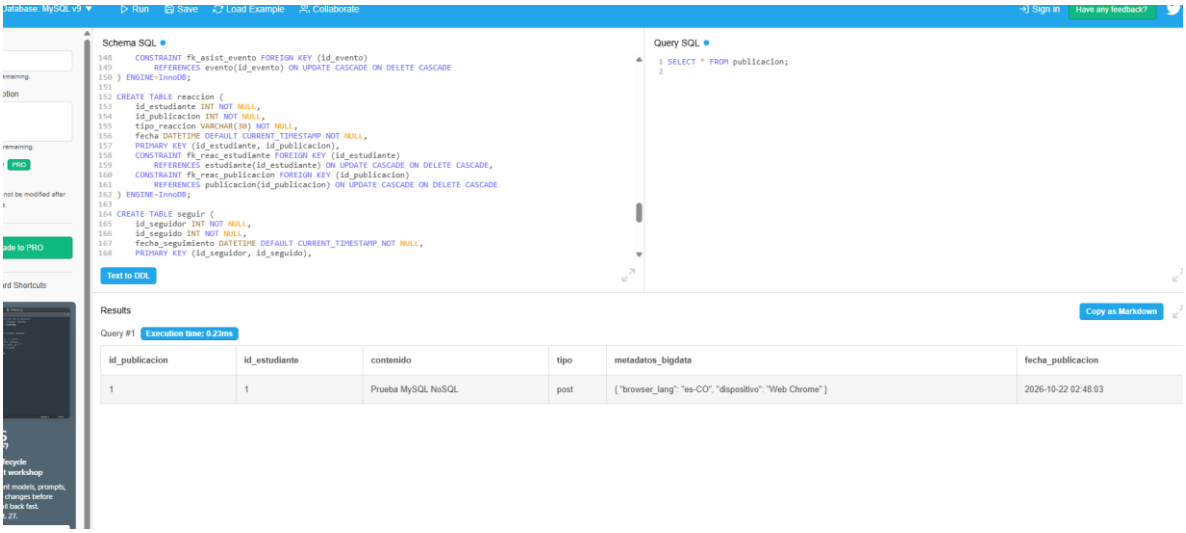
PostgreSQL 17



SQLite 3



MySQL 9.0


Database: MySQL v9
Run
Save
Load Example
Collaborate
Sign in
Have any feedback?
Schema SQL
148 CONSTRAINT fk_asist_evento FOREIGN KEY (id_evento)
149 REFERENCES evento(id_evento) ON UPDATE CASCADE ON DELETE CASCADE
150) ENGINE=InnoDB;
151
152 CREATE TABLE reaccion (
153 id_estudiante INT NOT NULL,
154 id_publicacion INT NOT NULL,
155 tipo_reaccion VARCHAR(30) NOT NULL,
156 fecha DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
157 PRIMARY KEY (id_estudiante, id_publicacion),
158 CONSTRAINT fk_reac_estudiante FOREIGN KEY (id_estudiante)
159 REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,
160 CONSTRAINT fk_reac_publicacion FOREIGN KEY (id_publicacion)
161 REFERENCES publicacion(id_publicacion) ON UPDATE CASCADE ON DELETE CASCADE
162) ENGINE=InnoDB;
163
164 CREATE TABLE seguir (
165 id_seguidor INT NOT NULL,
166 id_seguido INT NOT NULL,
167 fecha_seguimiento DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,
168 PRIMARY KEY (id_seguidor, id_seguido),
Text to DDL
Query SQL
1 SELECT * FROM publicacion;
2
Results
Copy as Markdown
Query #1
Execution time: 0.23ms
id_publicacion
id_estudiante
contenido
tipo
metadatos_bigdata
fecha_publicacion
1
1
Prueba MySQL NoSQL
post
{"browser_lang": "es-CO", "dispositivo": "Web Chrome"}
2026-10-22 02:48:03